

Journal of Functional Programming

Journal & vol/Article reference: JFP-2400016

Total number of pages (article pages only): 25

Date: 26-12-2024

This proof has been sent to you on behalf of Cambridge University Press. Please review this page and the proofs carefully, mark any corrections as necessary and answer all queries as listed on the 'Queries' page. Please note that this pdf is for proof checking purposes only. It should not be distributed to third parties and may not represent the final published version.

IMPORTANT: Please return the marked PDF proof within **5** days of receipt to:

cup.pm@luminad.com and jfp.production@cambridge.org

Authors are strongly advised to read these proofs thoroughly as any errors missed may appear in the final published paper. This will be your ONLY chance to correct your proof. Once published, either online or in print, no further changes can be made.

Instructions for supplying corrections:

Please ensure that you review the PDF document carefully and answer all author queries included on the following page. ALL CORRECTIONS/NOTES MUST BE ELECTRONICALLY MARKED DIRECTLY ON THE PDF PROOFS AND BE CLEAR AND SUCCINCT. Emails are not passed on with the PDF. Please do not provide lists, dialogue and long explanations. Do not attempt to edit the body of the text within the pdf in any way.

If you have no corrections to make, then please contact the email address as shown in the orange box above. Please ensure that you quote the Journal & Vol/Article reference (shown at the top of the page) and your full name.

Instructions for marking up electronically:

All proofs are enabled to allow electronic annotation using the 'Tools' function in the free Adobe Reader software. Full descriptions of the Adobe Reader tools are available at:

<https://helpx.adobe.com/uk/acrobat/using/commenting-pdfs.html>

Please do not use the 'Sticky note' function, as its placement is not precise enough. Please do not emphasise your marks in any way, 'Show comments' allows all marks to be clearly seen. Please return the file as an attachment via email to the address listed in the orange box above.

If you have any questions or difficulty, please contact the CUP Content Manager at:

jfp.production@cambridge.org

NOTES

This proof is sent to you for correction of **typographical errors only**. Revision of the substance of the text is not permitted, unless previously discussed with the editor of the journal.

- Only one set of corrections is permitted.
- Corrections which do NOT follow journal style will not be accepted.
- A new copy of a figure must be provided if correction of anything other than a typographical error introduced by the typesetter is required.
- You must return any forms included within your proof. We cannot publish your article if you have not returned a signed copyright/Open Access/licence to publish form. Further information regarding this can be found at www.cambridge.org/core/services/open-access-policies/open-access-journals

Author queries

- Q1: The distinction between surnames can be ambiguous, therefore to ensure accurate tagging for indexing purposes online (e.g. for PubMed entries), please check that the highlighted surnames have been correctly identified, that all names are in the correct order and spelt correctly.
- Q2: Please check that affiliations of all the authors and the corresponding author details are correctly set.
- Q3: AU: Figure citations were not sequential, so both citations and the respective figures have been renumbered according to the order in which they were cited. Please check and confirm this has been done correctly.

FUNCTIONAL PEARL

You could have invented Fenwick trees

BRENT YORGEY^{ib}

Hendrix College, 1600 Washington Ave, Conway, AR 72032, USA

(e-mail: yorgey@hendrix.edu)

Q1

Q2

Abstract

Fenwick trees, also known, are a clever solution to the problem of maintaining a sequence of values while allowing both updates and range queries in sublinear time. Their implementation is concise and efficient—but also somewhat baffling, consisting largely of nonobvious bitwise operations on indices. We begin with *segment trees*, a much more straightforward, easy-to-verify, purely functional solution to the problem, and use equational reasoning to explain the implementation of Fenwick trees as an optimized variant, making use of a Haskell EDSL for operations on infinite two’s complement binary numbers.

1 Introduction

Suppose we have a sequence of n integers $a_1, a_2, \dots, a_n$ and want to be able to perform arbitrary interleavings of the following two operations, as illustrated in Figure 1:

- *Update* the value at any given index¹ i by adding some value v .
- Find the sum of all values in any given range $[i, j]$, that is, $a_i + a_{i+1} + \dots + a_j$. We call this operation a *range query*.

Note that update is phrased in terms of *adding* some value v to the existing value; we can also *set* a given index to a new value v by adding $v - u$, where u is the old value.

If we simply store the integers in a mutable array, then we can update in constant time, but range queries require time linear in the size of the range, since we must iterate through the entire range $[i, j]$ to add up the values.

In order to improve the running time of range queries, we could try to cache (at least some of) the range sums. However, this must be done with care, since the cached sums must be kept up to date when updating the value at an index. For example, a straightforward approach would be to use an array P where P_i stores the prefix sum $a_1 + \dots + a_i$; P can be precomputed in linear time via a scan. Now range queries are fast: we can obtain $a_i + \dots + a_j$ in constant time by computing $P_j - P_{i-1}$ (for convenience we set $P_0 = 0$ so this works

¹ Note that we use 1-based indexing here and throughout the paper, that is, the first item in the sequence has index 1. The reasons for this choice will become clear later.

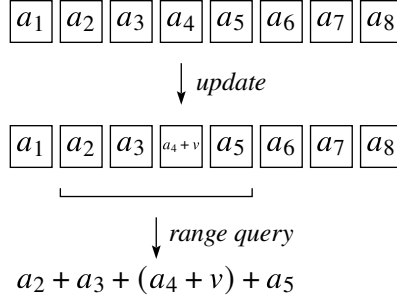


Fig. 1. Update and range query operations.

30 even when $i = 1$). Unfortunately, it is update that now takes linear time, since changing a_i
 31 requires updating P_j for every $j \geq i$.

32 Is it possible to design a data structure that allows *both* operations to run in sublin-
 33 ear time? (You may wish to pause and think about it before reading the next paragraph!)
 34 This is not just academic: the problem was originally considered in the context of *arith-*
 35 *metic coding* (Rissanen & Langdon, 1979; Bird & Gibbons, 2002), a family of techniques
 36 for turning messages into sequences of bits for storage or transmission. In order to min-
 37 imize the bits required, one generally wants to assign shorter bit sequences to more
 38 frequent characters, and vice versa; this leads to the need to maintain a dynamic table
 39 of character frequencies. We *update* the table every time a new character is processed
 40 and *query* the table for cumulative frequencies in order to subdivide a unit interval into
 41 consecutive segments proportional to the frequency of each character (Ryabko, 1989;
 42 Fenwick, 1994).

43 So, can we get both operations to run in sublinear time? The answer, of course, is yes.
 44 One simple technique is to divide the sequence into $\sqrt{n}$ buckets, each of size $\sqrt{n}$, and
 45 create an additional array of size $\sqrt{n}$ to cache the sum of each bucket. Updates still run in
 46 $O(1)$, since we simply have to update the value at the given index and the corresponding
 47 bucket sum. Range queries now run in $O(\sqrt{n})$ time: to find the sum $a_i + \dots + a_j$, we man-
 48 ually add the values from a_i to the end of its bucket, and from a_j to the beginning of its
 49 bucket; for all the buckets in between we can just look up their sum.

50 We can make range queries even faster, at the cost of making updates slightly slower, by
 51 introducing additional levels of caching. For example, we can divide the sequence into $\sqrt[3]{n}$
 52 “big buckets” and then further subdivide each big bucket into $\sqrt[3]{n}$ “small buckets”, with
 53 each small bucket holding $\sqrt[3]{n}$ values. The sum of each bucket is cached; now each update
 54 requires modifying three values, and range queries run in $O(\sqrt[3]{n})$ time.

55 In the limit, we end up with a binary divide-and-conquer approach to caching range
 56 sums, with both update and range query taking $O(\lg n)$ time. In particular, we can make a
 57 balanced binary tree where the leaves store the sequence itself, and every internal node
 58 stores the sum of its children. (This will be a familiar idea to many functional pro-
 59 grammers; for example, finger trees (Hinze & Paterson, 2006; Apfeldmus, 2009) use a
 60 similar sort of caching scheme.) The resulting data structure is popularly known as a

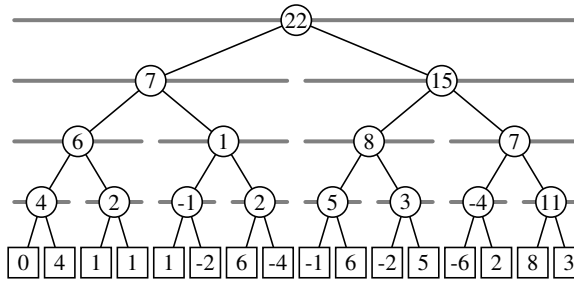


Fig. 2. A segment tree.

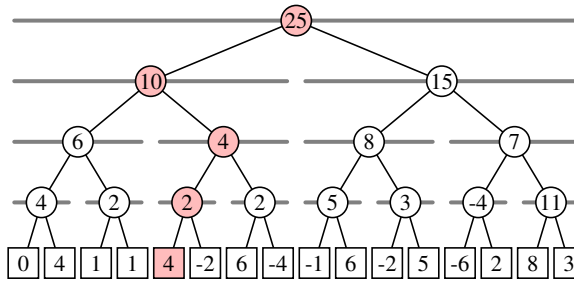


Fig. 3. Updating a segment tree.

61 *segment tree*,² presumably because each internal node ultimately caches the sum of a
 62 (contiguous) *segment* of the underlying sequence. Figure 2 shows a segment tree **Q3**
 63 on a sample array of length $n = 16$ (for simplicity, we will assume that n is a power of two,
 64 although it is easy to generalize to situations where it is not). Each leaf of the tree corre-
 65 sponds to an array entry; each internal node is drawn with a grey bar showing the segment
 66 of the underlying array of which it is the sum.

67 Let's see how we can use a segment tree to implement the two required operations so
 68 that they run in logarithmic time.

- 69 • To update the value at index i , we also need to update any cached range sums which
 70 include it. These are exactly the nodes along the path from the leaf at index i to the
 71 root of the tree; there are $O(\lg n)$ such nodes. Figure 3 illustrates this update process
 72 for the example segment tree from Figure 2; updating the entry at index 5 requires
 73 modifying only the shaded nodes along the path from the root to the updated entry.
- 74 • To perform a range query, we descend through the tree while keeping track of the
 75 range covered by the current node.
- 76 – If the range of the current node is wholly contained within the query range, return
 77 the value of the current node.

² There is some confusion of terminology here. As of this writing, the Wikipedia article on *segment trees* (Wikipedia Contributors, 2024) is about an interval data structure used in computational geometry. However, most of the Google search results for “segment tree” are from the world of competitive programming, where it refers to the data structure considered in this paper (see, for example, Halim *et al.*, 2020, Section 2.8 or Ivanov, 2011b). The two data structures are largely unrelated.

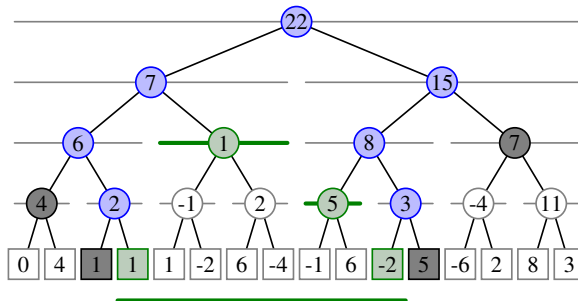


Fig. 4. Performing a range query on a segment tree.

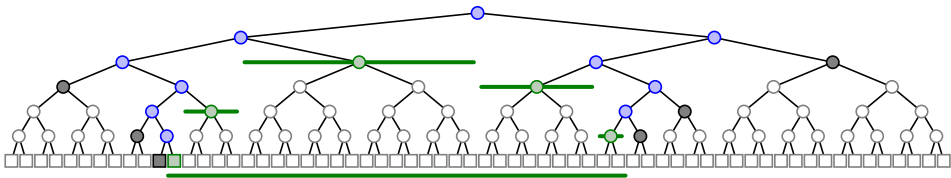


Fig. 5. Performing a range query on a larger segment tree.

78 – If the range of the current node is disjoint from the query range, return 0.
79 – Otherwise, recursively query both children and return the sum of the results.
80 Figure 4 illustrates the process of computing the sum of the range $[4 \dots 11]$. Blue
81 nodes are the ones we recurse through; green nodes are those whose range is wholly
82 contained in the query range and are returned without recursing further; grey nodes
83 are disjoint from the query range and return zero. The final result in this example is
84 the sum of values at the green nodes, $1 + 1 + 5 + -2 = 5$ (it is easily verified that this
85 is in fact the sum of values in the range $[4 \dots 11]$).
86 On this small example tree, it may seem that we visit a significant fraction of the
87 total nodes, but in general, we visit no more than about $4 \lg n$. Figure 5 makes this
88 more clear. Only one blue node in the entire tree can have two blue children, and
89 hence, each level of the tree can contain at most two blue nodes and two non-blue
90 nodes. We essentially perform two binary searches, one to find each endpoint of the
91 query range.

92 Segment trees are a very nice solution to the problem: as we will see in Section 2, they fit
93 well in a functional language; they also lend themselves to powerful generalizations such
94 as lazily propagated range updates and persistent update history via shared immutable
95 structure (Ivanov, 2011b).

96 *Fenwick trees*, or *binary indexed trees* (Fenwick, 1994; Ivanov, 2011a), are an alter-
97 native solution to the problem. What they lack in generality, they make up for with an
98 extremely small memory footprint—they require literally nothing more than an array stor-
99 ing the values in the tree—and a blazing fast implementation. In other words, they are
100 perfect for applications such as low-level coding/decoding routines where we don’t need
101 any of the advanced features that segment trees offer, and want to squeeze out every last
102 bit of performance.

```

class FenwickTree {
    private long[] a;
    public FenwickTree(int n) { a = new long[n+1]; }
    public long prefix(int i) {
        long s = 0;
        for (; i > 0; i -= LSB(i)) s += a[i]; return s;
    }
    public void update(int i, long delta) {
        for (; i < a.length; i += LSB(i)) a[i] += delta;
    }
    public long range(int i, int j) {
        return prefix(j) - prefix(i-1);
    }
    public long get(int i) { return range(i,i); }
    public void set(int i, long v) { update(i, v - get(i)); }
    private int LSB(int i) { return i & (-i); }
}

```

Fig. 6. Implementing Fenwick trees with bit tricks.

Figure 6 shows a typical implementation of Fenwick trees in Java. As you can see, the implementation is incredibly concise and consists mostly of some small loops doing just a few arithmetic and bit operations per iteration. It is not at all clear what this code is doing, or how it works! Upon closer inspection, the `range`, `get`, and `set` functions are straightforward, but the other functions are a puzzle. We can see that both the `prefix` and `update` functions call another function `LSB`, which for some reason performs a bitwise logical AND of an integer and its negation. In fact, `LSB(x)` computes the *least significant bit* of x , that is, it returns the smallest 2^k such that the k th bit of x is a one. However, it is not obvious how the implementation of `LSB` works, nor how and why least significant bits are being used to compute updates and prefix sums.

Our goal is *not* to write elegant functional code for this—already solved!—problem. Rather, our goal will be to use a functional domain-specific language for bit strings, along with equational reasoning, to *derive* and *explain* this baffling imperative code from first principles—a demonstration of the power of functional thinking and equational reasoning to understand code written even in other, non-functional languages. After developing more intuition for segment trees (Section 2), we will see how Fenwick trees can be viewed as a variant on segment trees (Section 3). We will then take a detour into two’s complement binary encoding, develop a suitable DSL for bit manipulations, and explain the implementation of the `LSB` function (Section 4). Armed with the DSL, we will then derive functions for converting back and forth between Fenwick trees and standard binary trees (Section 5). Finally, we will be able to derive functions for moving within a Fenwick tree by converting to binary tree indices, doing the obvious operations to effect the desired motion within the binary tree, and then converting back. Fusing away the conversions via equational reasoning will finally reveal the hidden `LSB` function, as expected (Section 6).

This paper was produced from a literate Haskell document; the source is available from GitHub, at <https://github.com/byorgey/fenwick/blob/master/Fenwick.lhs>.

2 Segment trees

Figure 7 exhibits a simple implementation of a segment tree in Haskell, using some utilities for working with index ranges shown in Figure 8. We store a segment tree as a recursive

```

data SegTree where
  Empty :: SegTree
  Branch :: Integer → Range → SegTree → SegTree → SegTree

  update :: Index → Integer → SegTree → SegTree
  update _ _ Empty = Empty
  update i v b@(Branch a rng l r)
    | i ∈ rng    = Branch (a + v) rng (update i v l) (update i v r)
    | otherwise = b

  rq :: Range → SegTree → Integer
  rq _ Empty = 0
  rq q (Branch a rng l r)
    | disjoint rng q = 0
    | rng ⊆ q        = a
    | otherwise      = rq q l + rq q r

  get :: Index → SegTree → Integer
  get i = rq (i :—: i)

  set :: Index → Integer → SegTree → SegTree
  set i v t = update i (v - get i t) t

```

Fig. 7. Simple segment tree implementation in Haskell.

```

type Index = Int
data Range = Index :—: Index -- (a :—: b) represents the closed interval [a, b]
deriving (Eq, Show)

(⊆) :: Range → Range → Bool
(lo1 :—: hi1) ⊆ (lo2 :—: hi2) = lo2 ≤ lo1 ∧ hi1 ≤ hi2

(∈) :: Index → Range → Bool
k ∈ i = (k :—: k) ⊆ i

disjoint :: Range → Range → Bool
disjoint (lo1 :—: hi1) (lo2 :—: hi2) = hi1 < lo2 ∨ hi2 < lo1

```

Fig. 8. Range utilities.

132 algebraic data type and implement *update* and *rq* using code that directly corresponds
 133 to the recursive descriptions given in the previous section; *get* and *set* can then also be
 134 implemented in terms of them. It is not hard to generalize this code to work for segment
 135 trees storing values from either an arbitrary commutative monoid if we don't need the *set*
 136 operation—or from an arbitrary Abelian group (i.e. commutative monoid with inverses) if
 137 we do need *set*—but we keep things simple since the generalization doesn't add anything
 138 to our story.

139 Although this implementation is simple and relatively straightforward to understand,
 140 compared to simply storing the sequence of values in an array, it incurs a good deal of
 141 overhead. We can be more clever in our use of space by storing all the nodes of a segment
 142 tree in an array, using the standard left-to-right breadth-first indexing scheme illustrated in
 143 Figure 9 (for example, this scheme, or something like it, is commonly used to implement
 144 binary heaps). The root has label 1; every time we descend one level we append an extra

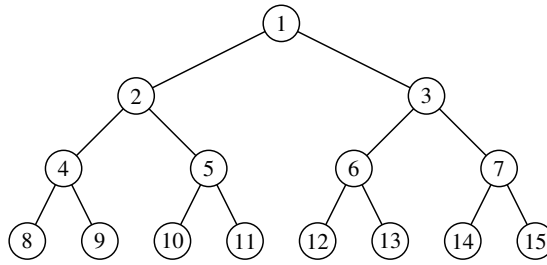


Fig. 9. Indexing a binary tree.

145 bit: 0 when we descend to the left child and 1 when we descend to the right. Thus, the
 146 index of each node expressed in binary records the sequence of left-right choices along the
 147 path to that node from the root. Going from a node to its children is as simple as doing
 148 a left bit shift and optionally adding 1; going from a node to its parent is a right bit shift.
 149 This defines a bijection from the positive natural numbers to the nodes of an infinite binary
 150 tree. If we label the segment tree array with $s_1 \dots s_{2n-1}$, then s_1 stores the sum of all the
 151 a_i , s_2 stores the sum of the first half of the a_i , s_3 stores the sum of the second half, and so
 152 on. $a_1 \dots a_n$ themselves are stored as $s_n \dots s_{2n-1}$.

153 The important point is that since descending recursively through the tree corresponds to
 154 simple operations on indices, all the algorithms we have discussed can be straightforwardly
 155 transformed into code that works with a (mutable) array: for example, instead of storing a
 156 reference to the current subtree, we store an integer index; every time we want to descend
 157 to the left or right, we simply double the current index or double and add one, and so on.
 158 Working with tree nodes stored in an array presents an additional opportunity: rather than
 159 being forced to start at the root and recurse downwards, we can start at a particular index
 160 of interest and move *up* the tree instead.

161 So how do we get from segment trees to Fenwick trees? We start with an innocuous-
 162 seeming observation: *not all the values stored in a segment tree are necessary*. Of course,
 163 all the non-leaf nodes are “unnecessary” in the sense that they represent cached range sums
 164 which could easily be recomputed from the original sequence. That’s the whole point:
 165 caching these “redundant” sums trades off space for time, allowing us to perform arbitrary
 166 updates and range queries quickly, at the cost of doubling the required storage space.

167 But that’s not what I mean! In fact, there is a different set of values we can forget about,
 168 but in such a way that we still retain the logarithmic running time for updates and range
 169 queries. Which values, you ask? Simple: just forget the data stored in *every node which*
 170 *is a right child*. Figure 10 shows the same example tree we have been using, but with the
 171 data deleted from every right child. Note that “every right child” includes both leaves and
 172 internal nodes: we forget the data associated to *every* node which is the right child of its
 173 parent. We will refer to the nodes with discarded data as *inactive* and the remaining nodes
 174 (that is, left children and the root) as *active*. We also say that a tree with all its right children
 175 inactivated in this way has been *thinned*.

176 Updating a thinned segment tree is easy: just update the same nodes as before, ignoring
 177 any updates to inactive nodes. But how do we answer range queries? It’s not too hard
 178 to see that there is enough information remaining to reconstruct the information that was

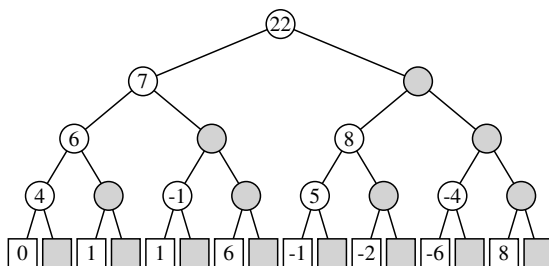


Fig. 10. Inactivating all right children in a segment tree.

discarded (you might like to try convincing yourself of this: can you deduce what values must go in the greyed-out nodes in Figure 10, without peeking at any previous figures?). However, in and of itself, this observation does not give us a nice algorithm for computing range sums.

It turns out the key is to think about *prefix sums*. As we saw in the introduction and the implementation of *range* in Figure 6, if we can compute the prefix sum $P_k = a_1 + \dots + a_k$ for any k , then we can compute the range sum $a_i + \dots + a_j$ as $P_j - P_{i-1}$.

Theorem 1. *Given a thinned segment tree, the sum of any prefix of the original array (and hence also any range sum) can be computed, in logarithmic time, using only the values of active nodes.*

Proof Surprisingly, in the special case of prefix queries, the original range query algorithm described in Section 1 and implemented in Figure 7 works unchanged! That is to say, the base case in which the range of the current node is wholly contained within the query range—and we thus return the value of the current node—will only ever happen at active nodes.

First, the root itself is active, and hence, querying the full range will work. Next, consider the case where we are at a node and recurse on both children. The left child is always active, so we only need to consider the case where we recurse to the right. It is impossible that the range of the right child will be wholly contained in the query range: since the query range is always a prefix of the form $[1, j]$, if the right child’s range is wholly contained in $[1, j]$ then the left child’s range must be as well—which means that the parent node’s range (which is the union of its children’s ranges) would also be wholly contained in the query range. But in that case we would simply return the parent’s value without recursing into the right child. Thus, when we do recurse into a right child, we might end up returning 0, or we might recurse further into both grandchildren, but in any case we will never try to look at the value of the right child itself. ■

Figure 11 illustrates performing a prefix query on a segment tree. Notice that visited right children are only ever blue or grey; the only green nodes are left children.

3 Fenwick trees

How should we actually store a thinned segment tree in memory? If we stare at Figure 10 again, one strategy suggests itself: simply take every active node and “slide” it down and

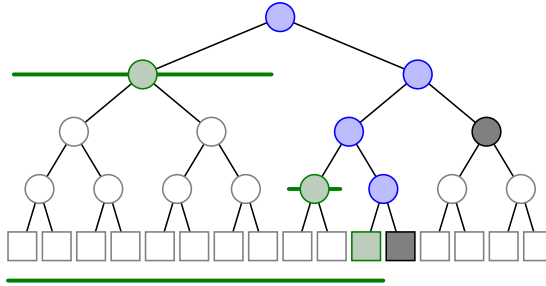


Fig. 11. Performing a prefix query on a segment tree.

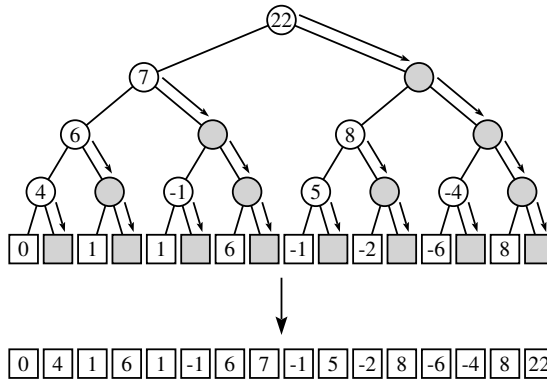


Fig. 12. Sliding active values down a thinned segment tree.

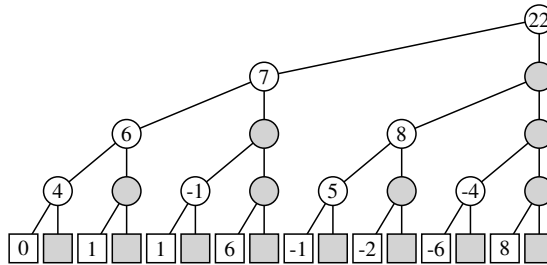


Fig. 13. Right-leaning drawing of a thinned segment tree, vertically aligning nodes with their storage location.

210 to the right until it lands in an empty slot in the underlying array, as illustrated in Figure 12.
 211 This sets up a one-to-one correspondence between active nodes and indices in the range
 212 $1 \dots n$. Another way to understand this indexing scheme is to use a postorder traversal
 213 of the tree, skipping over inactive nodes and giving consecutive indices to active nodes
 214 encountered during the traversal. We can also visualize the result by drawing the tree in a
 215 “right-leaning” style (Figure 13), vertically aligning each active node with the array slot
 216 where it is stored.

217 This method of storing the active nodes from a thinned segment tree in an array is pre-
 218 cisely a *Fenwick tree*. I will also sometimes refer to it as a *Fenwick array*, when I want to

219 particularly emphasize the underlying array data structure. Although it is certainly a clever
 220 use of space, the big question is how to implement the update and range query operations.
 221 Our implementations of these operations for segment trees worked by recursively descend-
 222 ing through the tree, either directly if the tree is stored as a recursive data structure, or using
 223 simple operations on indices if the tree is stored in an array. However, when storing the
 224 active nodes of a thinned tree in a Fenwick array, it is not *a priori* obvious what operations
 225 on array indices will correspond to moving around the tree. In order to attack this problem,
 226 we first take a detour through a domain-specific language for two’s complement binary
 227 values.

228 4 Two’s complement binary

229 The bit tricks usually employed to implement Fenwick trees rely on a *two’s complement*
 230 representation of binary numbers, which allow positive and negative numbers to be repre-
 231 sented in a uniform way; for example, a value consisting of all 1 bits represents -1 . We
 232 therefore turn now to developing a domain-specific language, embedded in Haskell, for
 233 manipulating two’s complement binary representations.

234 First, we define a type of bits, with functions for inversion, logical conjunction, and
 235 logical disjunction:

236 **data** *Bit* = *O* | *I* **deriving** (*Eq*, *Ord*, *Show*, *Enum*)

237 $\neg :: \text{Bit} \rightarrow \text{Bit}$

238 $\neg = \lambda \text{case} \{ O \rightarrow I; I \rightarrow O \}$

239 $(\wedge), (\vee) :: \text{Bit} \rightarrow \text{Bit} \rightarrow \text{Bit}$

240 $O \wedge _ = O$

241 $I \wedge b = b$

242 $I \vee _ = I$

243 $O \vee b = b$

244 Next, we must define bit strings, i.e. sequences of bits. Rather than fix a specific bit
 245 width, it will be much more elegant to work with *infinite* bit strings.³ It is tempting to use
 246 standard Haskell lists to represent potentially infinite bit strings, but this leads to a number
 247 of problems. For example, equality of infinite lists is not decidable, and there is no way
 248 in general to convert from an infinite list of bits back to an *Integer*—how would we know
 249 when to stop? In fact, these practical problems stem from a more fundamental one: infinite
 250 lists of bits are actually a bad representation for two’s complement bit strings, because
 251 of “junk”, that is, infinite lists of bits which do not correspond to values in our intended
 252 semantic domain. For example, *cycle* [*I*, *O*] is an infinite list which alternates between *I*
 253 and *O* forever, but it does not represent a valid two’s complement encoding of an integer.
 254 Even worse are non-periodic lists, such as the one with *I* at every prime index and *O*
 255 everywhere else.

256 In fact, the bit strings we want are the *eventually constant* ones, that is, strings which
 257 eventually settle down to an infinite tail of all zeros (which represent nonnegative integers)

³ Some readers may recognize infinite two’s complement bit strings as 2-*adic* numbers, that is, *p*-*adic* numbers for the specific case $p = 2$, but nothing in our story depends on understanding the connection.

258 or all ones (which represent negative integers). Every such string has a finite representation,
 259 so directly encoding eventually constant bit strings in Haskell not only gets rid of the junk
 260 but also leads to elegant, terminating algorithms for working with them.

261 **data Bits where**

262 $Rep :: Bit \rightarrow Bits$

263 $Snoc :: !Bits \rightarrow Bit \rightarrow Bits$

264 $Rep\ b$ represents an infinite sequence of bit b , whereas $Snoc\ bs\ b$ represents the bit string
 265 bs followed by a final bit b . We use $Snoc$, rather than $Cons$, to match the way we usually
 266 write bit strings, with the least significant bit last. Note also the use of a *strictness annotation*
 267 on the $Bits$ field of $Snoc$; this is to rule out infinite lists of bits using only $Snoc$, such as
 268 $bs = Snoc\ (Snoc\ bs\ 0)\ I$. In other words, the only way to make a non-bottom value of type
 269 $Bits$ is to have a finite sequence of $Snoc$ finally terminated by Rep .

270 Although we have eliminated junk values, one remaining problem is that there can
 271 be multiple distinct representations of the same value. For example, $Snoc\ (Rep\ 0)\ 0$ and
 272 $Rep\ 0$ both represent the infinite bit string containing all zeros. However, we can solve this
 273 with a carefully constructed *bidirectional pattern synonym* (Pickering *et al.*, 2016).

274 $toSnoc :: Bits \rightarrow Bits$

275 $toSnoc\ (Rep\ a) = Snoc\ (Rep\ a)\ a$

276 $toSnoc\ as = as$

277 $pattern\ (:.) :: Bits \rightarrow Bit \rightarrow Bits$

278 $pattern\ (:.)\ bs\ b \leftarrow (toSnoc \rightarrow Snoc\ bs\ b)$

279 **where**

280 $Rep\ b :: b' \mid b \equiv b' = Rep\ b$

281 $bs :: b = Snoc\ bs\ b$

282 $\{-\# \text{ COMPLETE } (:.) \#-\}$

283 Matching with the pattern $(bs :: b)$ uses a *view pattern* (Erwig & Jones, 2001) to poten-
 284 tially expand a Rep one step into a $Snoc$, so that we can pretend $Bits$ values are always
 285 constructed with $(:.)$. Conversely, constructing a $Bits$ with $(:.)$ will do nothing if we hap-
 286 pen to snoc an identical bit b onto an existing $Rep\ b$. This ensures that as long as we stick
 287 to using $(:.)$ and never directly use $Snoc$, $Bits$ values will always be *normalized* so that the
 288 terminal $Rep\ b$ is immediately followed by a different bit. Finally, we mark the pattern $(:.)$
 289 as COMPLETE on its own, since matching on $(:.)$ is indeed sufficient to handle every pos-
 290 sible input of type $Bits$. However, in order to obtain terminating algorithms we will often
 291 include one or more special cases for Rep .

292 Let's begin with some functions for converting $Bits$ to and from $Integer$ and for
 293 displaying $Bits$ (intended only for testing).

294 $toBits :: Int \rightarrow Bits$

295 $toBits\ n$

296 $\mid n \equiv 0 = Rep\ 0$

297 $\mid n \equiv -1 = Rep\ 1$

298 $\mid otherwise = toBits\ (n \div 2) :: toEnum\ (n \bmod 2)$

299 $fromBits :: Bits \rightarrow Int$

```

300 fromBits (Rep O) = 0
301 fromBits (Rep I) = -1
302 fromBits (bs :: b) = 2 · fromBits bs + fromEnum b
303 instance Show Bits where
304   show = reverse ∘ go
305   where
306     go (Rep b) = replicate 3 (showBit b) ++ "..."
307     go (bs :: b) = showBit b : go bs
308     showBit = ("01"!!) ∘ fromEnum
309   Let's try it out, using QuickCheck (Claessen & Hughes, 2000) to verify our conversion
310   functions:

```

```

ghci> Rep 0 :: 0 :: I :: 0 :: I
...000101
ghci> Rep I :: 0 :: I
...11101
ghci> toBits 26
...00011010
ghci> toBits (-30)
...11100010
ghci> fromBits (toBits (-30))
-30
ghci> quickCheck $ \x -> fromBits (toBits x) == x
+++ OK, passed 100 tests.

```

311 We can now begin implementing some basic operations on *Bits*. First, incrementing and
312 decrementing can be implemented recursively as follows:

```

313 inc :: Bits → Bits
314 inc (Rep I) = Rep O
315 inc (bs :: O) = bs :: I
316 inc (bs :: I) = inc bs :: O
317 dec :: Bits → Bits
318 dec (Rep O) = Rep I
319 dec (bs :: I) = bs :: O
320 dec (bs :: O) = dec bs :: I

```

321 The *least significant bit*, or LSB, of a sequence of bits can be defined as follows:

```

322 lsb :: Bits → Bits
323 lsb (Rep O) = Rep O
324 lsb (bs :: O) = lsb bs :: O
325 lsb (_ :: I) = Rep O :: I

```

326 Note that we add a special case for *Rep O* to ensure that *lsb* is total. Technically, *Rep O*
327 does not have a least significant bit, so defining *lsb (Rep O) = Rep O* seems sensible.

```

ghci> toBits 26
"...00011010"
ghci> lsb $ toBits 26
"...00010"
ghci> toBits 24
"...00011000"
ghci> lsb $ toBits 24
"...0001000"

```

328 Bitwise logical conjunction can be defined straightforwardly. Note that we only need
 329 two cases; if the finite parts of the inputs have different lengths, matching with $(:.)$ will
 330 automatically expand the shorter one to match the longer one.

```

331 ( $\odot$ ) :: Bits  $\rightarrow$  Bits  $\rightarrow$  Bits
332 Rep x  $\odot$  Rep y = Rep (x  $\wedge$  y)
333 (xs  $:. x$ )  $\odot$  (ys  $:. y$ ) = (xs  $\odot$  ys)  $:. (x \wedge y)$ 

```

334 Bitwise inversion is likewise straightforward.

```

335 inv :: Bits  $\rightarrow$  Bits
336 inv (Rep b) = Rep ( $\neg$  b)
337 inv (bs  $:. b$ ) = inv bs  $:. \neg b$ 

```

338 The above functions follow familiar patterns. We could easily generalize to eventually
 339 constant streams over an arbitrary element type and then implement $(\odot)$ in terms of a
 340 generic *zipWith* and *inv* in terms of *map*. However, for the present purpose we do not need
 341 the extra generality.

342 We implement addition with the usual carry-propagation algorithm, along with some
 343 special cases for *Rep*.

```

344 ( $\oplus$ ) :: Bits  $\rightarrow$  Bits  $\rightarrow$  Bits
345 xs       $\oplus$  Rep O    = xs
346 Rep O    $\oplus$  ys      = ys
347 Rep I    $\oplus$  Rep I   = Rep I  $:. O$ 
348 Snoc xs I  $\oplus$  Snoc ys I = inc (xs  $\oplus$  ys)  $:. O$ 
349 Snoc xs x  $\oplus$  Snoc ys y = (xs  $\oplus$  ys)  $:. (x \vee y)$ 

```

350 It is not too hard to convince ourselves that this definition of addition is terminating and
 351 yields correct results; but we can also be fairly confident by just trying it with QuickCheck:

```

ghci> quickCheck $ \x y -> fromBits (toBits x  $+. toBits y$ ) == x + y
+++ OK, passed 100 tests.

```

352 Finally, the following definition of negation is probably familiar to anyone who has
 353 studied two's complement arithmetic; I leave it as an exercise for the interested reader to
 354 prove that $x \oplus \text{neg } x \equiv \text{Rep } O$ for all $x :: \text{Bits}$.

```

355 neg :: Bits  $\rightarrow$  Bits
356 neg = inc  $\circ$  inv

```

357 We now have the tools to resolve the first mystery of the Fenwick tree implementation.

358 **Theorem 4.1.** *For all $x :: \text{Bits}$,*

$$\text{lsb } x = x \oplus \text{neg } x.$$

359 **Proof** By induction on x .

- 360 • First, if $x = \text{Rep } O$, it is an easy calculation to verify that $\text{lsb } x = x \oplus \text{neg } x = \text{Rep } O$.
- 361 • Likewise, if $x = \text{Rep } I$, both $\text{lsb } x$ and $x \oplus \text{neg } x$ reduce to $\text{Rep } O :: I$.
- 362 • If $x = xs :: O$, then $\text{lsb } x = \text{lsb } (xs :: O) = \text{lsb } xs :: O$ by definition, whereas

$$\begin{aligned}
 & (xs :: O) \oplus \text{neg } (xs :: O) \\
 = & \quad \{ \text{Definition of } \text{neg} \} \\
 & (xs :: O) \oplus \text{inc } (\text{inv } (xs :: O)) \\
 = & \quad \{ \text{Definition of } \text{inv} \text{ and } \neg \} \\
 & (xs :: O) \oplus \text{inc } (\text{inv } xs :: I) \\
 = & \quad \{ \text{Definition of } \text{inc} \} \\
 & (xs :: O) \oplus (\text{inc } (\text{inv } xs) :: O) \\
 = & \quad \{ \text{Definition of } \oplus \text{ and } \text{neg} \} \\
 & (xs \oplus \text{neg } xs) :: O \\
 = & \quad \{ \text{Induction hypothesis} \} \\
 & \text{lsb } xs :: O
 \end{aligned}$$

- 363 • Next, if $x = xs :: I$, then $\text{lsb } (xs :: I) = \text{Rep } O :: I$ by definition, whereas

$$\begin{aligned}
 & (xs :: I) \oplus \text{neg } (xs :: I) \\
 = & \quad \{ \text{Definition of } \text{neg} \} \\
 & (xs :: I) \oplus \text{inc } (\text{inv } (xs :: I)) \\
 = & \quad \{ \text{Definition of } \text{inv} \text{ and } \neg \} \\
 & (xs :: I) \oplus \text{inc } (\text{inv } xs :: O) \\
 = & \quad \{ \text{Definition of } \text{inc} \} \\
 & (xs :: I) \oplus (\text{inv } xs :: I) \\
 = & \quad \{ \text{Definition of } \oplus \} \\
 & (xs \oplus \text{inv } xs) :: I \\
 = & \quad \{ \text{Bitwise AND of } xs \text{ and its inverse is } \text{Rep } O \} \\
 & \text{Rep } O :: I
 \end{aligned}$$

364

365 For the last equality, we need a lemma that $xs \oplus \text{inv } xs = \text{Rep } O$, which should be
 366 intuitively clear and can easily be proved by induction as well.

367 Finally, in order to express the index conversion functions we will develop in the next
 368 section, we need a few more things in our DSL. First, some functions to set and clear
 369 individual bits and to test whether particular bits are set:

370 $\text{setTo} :: \text{Bit} \rightarrow \text{Int} \rightarrow \text{Bits} \rightarrow \text{Bits}$

371 $\text{setTo } b' \ 0 \ (bs :: _) = bs :: b'$

372 $\text{setTo } b' \ k \ (bs :: b) = \text{setTo } b' \ (k - 1) \ bs :: b$

```

373 set, clear :: Int → Bits → Bits
374 set = setTo I
375 clear = setTo O
376 test :: Int → Bits → Bool
377 test 0 (bs :. b) = b ≡ I
378 test n (bs :. _) = test (n − 1) bs
379 even, odd :: Bits → Bool
380 odd = test 0
381 even = not ∘ odd

```

382 The only other things we will need are left and right shift, and a generic *while* combinator
 383 that iterates a given function, returning the first iterate for which a predicate is false.

```

384 shr :: Bits → Bits
385 shr (bs :. _) = bs
386 shl :: Bits → Bits
387 shl = (: O)
388 while :: (a → Bool) → (a → a) → a → a
389 while pf x
390   | px          = while pf (f x)
391   | otherwise = x

```

392 5 Index conversion

393 Before deriving our index conversion functions, we must deal with one slightly awkward
 394 fact. In a traditional binary tree indexing scheme, as shown in Figure 9, the root has index
 395 1, every left child is twice its parent, and every right child is one more than twice its parent.
 396 Recall that in a thinned segment tree, the root node and every left child are active, with all
 397 right children being inactive. This makes the root an awkward special case—all active
 398 nodes have an even index, *except* the root, which has index 1. This makes it more difficult
 399 to check whether we are at an active node—it is not enough to simply look at the least
 400 significant bit.

401 One easy way to fix this is simply to give the root index 2 and then proceed to label the
 402 rest of the nodes using the same scheme—every left child is twice its parent, and every right
 403 child is one more than twice its parent. This results in the indexing shown in Figure 14, as
 404 if we had just taken the left subtree of the tree rooted at 1, and ignored the right subtree. Of
 405 course, this means about half the possible indices are omitted—but that’s not a problem,
 406 since we will only use these indices as an intermediate step which will eventually get fused
 407 away.

408 Figure 15 shows a binary tree where nodes have been numbered in two different ways:
 409 the left side of each node shows the node’s binary tree index (with the root having index 2).
 410 The right side of each node shows its index in the Fenwick array, if it has one (inactive
 411 nodes simply have their right half greyed out). The table underneath shows the mapping
 412 from Fenwick array indices (top row) to binary tree indices (bottom row). As a larger
 413 example, Figure 16 shows the same thing on a binary tree one level deeper.

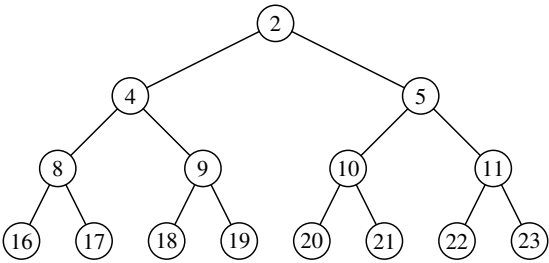


Fig. 14. Indexing a binary tree with 2 at the root.

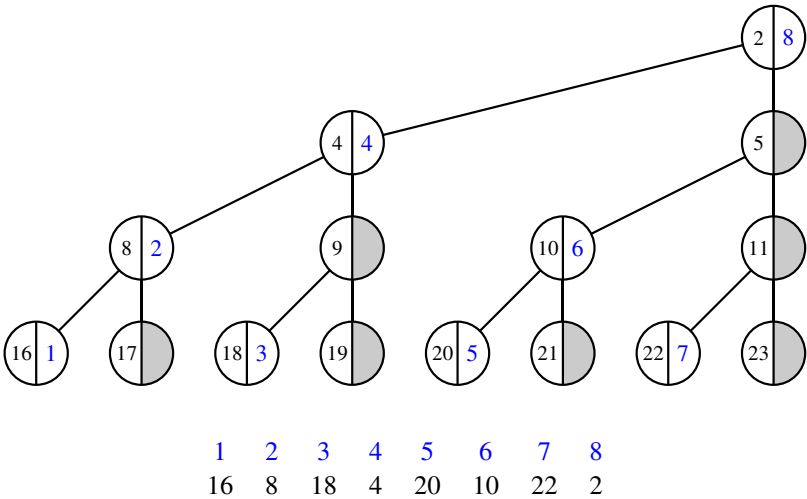


Fig. 15. Binary tree labelled with both binary and Fenwick indexing.

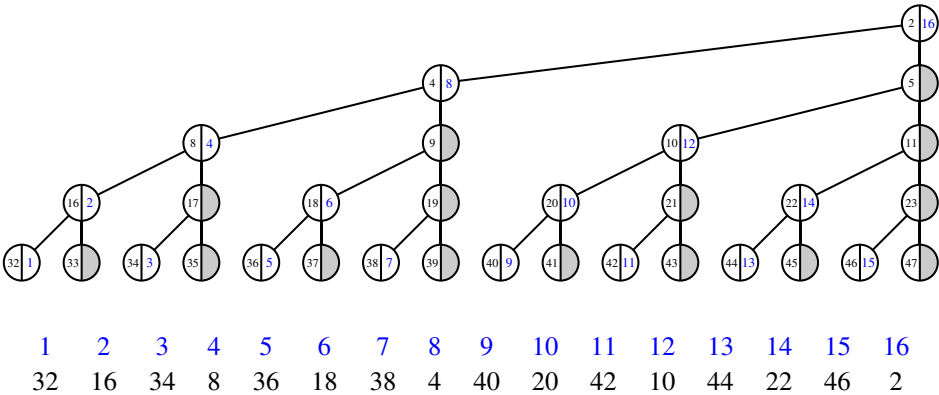


Fig. 16. Binary tree labelled with both binary and Fenwick indexing.

414 Our goal is to come up with a way to calculate the binary index for a given Fenwick
415 index or vice versa. Staring at the table in Figure 16, a few patterns stand out. Of course,
416 all the numbers in the bottom row are even, which is precisely because the binary tree
417 is numbered in such a way that all active nodes have an even index. Second, we can see
418 the even numbers 32, 34 . . . 46, in order, in all the odd positions. These are exactly the
419 leaves of the tree, and indeed, every other node in the Fenwick array will be a leaf from the

$$\begin{aligned}
(\vee) &:: [a] \rightarrow [a] \rightarrow [a] \\
[] &\vee _ = [] \\
(x : xs) &\vee ys = x : (ys \vee xs) \\
b &:: Int \rightarrow [Int] \\
b\ 0 &= [2] \\
b\ n &= \text{map}\ (2 \cdot) [2^n \dots 2^n + 2^{n-1} - 1] \vee b\ (n - 1)
\end{aligned}$$

Fig. 17. Recurrence for sequence of binary tree indices in a Fenwick array.

$$\begin{aligned}
(a : _) &! 1 = a \\
(_ : as) &! k = as ! (k - 1) \\
-- \text{ If } |xs| &\equiv |ys|: \\
(xs \vee ys) &! (2 \cdot j) = ys ! j \\
(xs \vee ys) &! (2 \cdot j - 1) = xs ! j
\end{aligned}$$

Fig. 18. Indexing and interleaving.

420 original tree. Alternating with these, in the even positions, are the numbers 16 8 18 4 . . . ,
 421 which correspond to all the non-leaf nodes; but these are exactly the sequence of binary
 422 indices from the bottom row of the table in Figure 15—since the internal nodes in a tree
 423 of height 4 themselves constitute a tree of height 3, with the nodes occurring in the same
 424 order.

425 These observations lead to the recurrence shown in Figure 17 for the sequence b_n of
 426 binary indices for the nodes stored in a Fenwick array of length 2^n : b_0 is just the single-
 427 ton sequence $[2]$, and otherwise b_n is the even numbers $2^{n+1}, 2^{n+1} + 2, \dots, 2^{n+1} + 2^n - 2$
 428 interleaved with b_{n-1} .

429 We can check that this does in fact reproduce the observed sequence for $n = 4$:

```
ghci> b 4
[32, 16, 34, 8, 36, 18, 38, 4, 40, 20, 42, 10, 44, 22, 46, 2]
```

430 Let $s!k$ denote the k th item in the list s (counting from 1), as defined in Figure 18.
 431 The same figure also lists two easy lemmas about the interaction between indexing and
 432 interleaving, namely $(xs \vee ys)!(2 \cdot j) = ys!j$ and $(xs \vee ys)!(2 \cdot j - 1) = xs!j$ (as long as xs and
 433 ys have equal lengths). With these in hand, we can define the Fenwick to binary index
 434 conversion function as

$$f2b\ n\ k = b\ n!k.$$

435 Of course, since b_n is of length 2^n , this function is only defined on the range $[1, 2^n]$.

436 We can now simplify the definition of $f2b$ as follows. First of all, for even inputs, we
 437 have

$$\begin{aligned}
&f2b\ n\ (2 \cdot j) \\
= & \quad \{ \text{Definition of } f2b \} \\
&b\ n!(2 \cdot j) \\
= & \quad \{ \text{Definition of } b \}
\end{aligned}$$

$$\begin{aligned}
& (\text{map } (2 \cdot) [2^n \dots 2^n + 2^{n-1} - 1] \vee b (n-1))! (2 \cdot j) \\
= & \quad \{ \quad \forall \neg \mathcal{W} \text{ lemma} \quad \} \\
& b(n-1) \mathcal{W} j \\
= & \quad \{ \quad \text{Definition of } f2b \quad \} \\
& f2b (n-1) j.
\end{aligned}$$

438 And for odd inputs,

$$\begin{aligned}
& f2b n (2 \cdot j - 1) \\
= & \quad \{ \quad \text{Definition of } f2b \quad \} \\
& b n! (2 \cdot j - 1) \\
= & \quad \{ \quad \text{Definition of } b \quad \} \\
& (\text{map } (2 \cdot) [2^n \dots 2^n + 2^{n-1} - 1] \vee b (n-1))! (2 \cdot j - 1) \\
= & \quad \{ \quad \forall \neg \mathcal{W} \text{ lemma} \quad \} \\
& \text{map } (2 \cdot) [2^n \dots 2^n + 2^{n-1} - 1] j \\
= & \quad \{ \quad \text{Definition of } \text{map}, \text{ algebra} \quad \} \\
& 2 \cdot (2^n + j - 1) \\
= & \quad \{ \quad \text{algebra} \quad \} \\
& 2^{n+1} + 2j - 2
\end{aligned}$$

439 Thus, we have

$$f2b n k = \begin{cases} f2b (n-1) (k / 2) & k \text{ even} \\ 2^{n+1} + k - 1 & k \text{ odd} \end{cases}$$

440 Note that when $n = 0$, we must have $k = 1$, and hence, $f2b 0 1 = 2^0 + 1 - 1 = 1$, as required,
441 so this definition is valid for all $n \geq 0$. Now factor k uniquely as $2^a \cdot b$ where b is odd. Then
442 by induction we can see that

$$f2b n (2^a \cdot b) = f2b (n-a) b = 2^{n-a+1} + b - 1.$$

443 So, in other words, computing $f2b$ consists of repeatedly dividing by 2 (i.e. right bit shifts)
444 as long as the input is even and then finally decrementing and adding a power of 2.
445 However, knowing what power of 2 to add at the end depends on knowing how many
446 times we shifted. A better way to think of it is to add 2^{n+1} at the *beginning*, and then let it
447 be shifted along with everything else. Thus, we have the following definition of $f2b'$ using
448 our *Bits* DSL. Defining $\text{shift } n = \text{while even shr} \circ \text{set } n$ separately will make some of our
449 proofs more compact later.

450 $\text{shift} :: \text{Int} \rightarrow \text{Bits} \rightarrow \text{Bits}$
451 $\text{shift } n = \text{while even shr} \circ \text{set } n$
452 $f2b' :: \text{Int} \rightarrow \text{Bits} \rightarrow \text{Bits}$
453 $f2b' n = \text{dec} \circ \text{shift } (n+1)$

454 For example, we can verify that this produces identical results to $f2b 4$ on the range
455 $[1, 2^4]$ (for convenience, we define $(f == g) k = f k \equiv g k$):

```

456 ghci> all (f2b 4 == fromBits . f2b' 4 . toBits) [1 .. 2^4]
457 True

```

458 We now turn to deriving $b2f\ n$, which converts back from binary to Fenwick indices.
 459 $b2f\ n$ should be a left inverse to $f2b\ n$, that is, for any $k \in [1, 2^n]$ we should have
 460 $b2f\ n (f2b\ n\ k) \equiv k$. If k is an input to $f2b$, we have $k = 2^a \cdot b \leq 2^n$, and so $b - 1 < 2^{n-a}$.
 461 Hence, given the output $f2b\ n\ k = m = 2^{n-a+1} + b - 1$, the highest bit of m is 2^{n-a+1} , and the
 462 rest of the bits represent $b - 1$. So, in general, given some m which is the output of $f2b\ n$,
 463 we can write it uniquely as $m = 2^c + d$ where $d < 2^{c-1}$; then

$$b2f\ n (2^c + d) = 2^{n-c+1} \cdot (d + 1).$$

464 In other words, given the input $2^c + d$, we subtract off the highest bit 2^c , increment, then
 465 left shift $n - c + 1$ times. Again, though, there is a simpler way: we can increment first
 466 (note since $d < 2^{c-1}$, incrementing cannot disturb the bit at 2^c), then left shift enough times
 467 to bring the leftmost bit into position $n + 1$, and finally remove it. That is:

```

468 unshift :: Int → Bits → Bits
469 unshift n = clear n ∘ while (not ∘ test n) shl
470 b2f' :: Int → Bits → Bits
471 b2f' n = unshift (n + 1) ∘ inc
472 Verifying:

```

```

ghci> all (fromBits . b2f' 4 . f2b' 4 . toBits == id) [1 .. 2^4]
True

```

473 6 Deriving Fenwick operations

474 We can now finally derive the required operations on Fenwick array indices for moving
 475 through the tree, by starting with operations on a binary indexed tree and conjugating
 476 by conversion to and from Fenwick indices. First, in order to fuse away the resulting
 477 conversion, we will need a few lemmas.

478 **Lemma 6.1** (shr-inc-dec). *For all $bs :: Bits$ which are odd (that is, end with 1),*

- 479 • $(shr \circ dec)\ bs = shr\ bs$
- 480 • $(shr \circ inc)\ bs = (inc \circ shr)\ bs$

481 **Proof** Both are immediate by definition. ■

482 **Lemma 6.2** (while-inc-dec). *The following both hold for all $Bits$ values:*

- 483 • $inc \circ while\ odd\ shr = while\ even\ shr \circ inc$
- 484 • $dec \circ while\ even\ shr = while\ odd\ shr \circ dec$

485 **Proof** Easy proof by induction on *Bits*. For example, for the *inc* case, the functions on
 486 both sides discard consecutive 1 bits and then flip the first 0 bit to a 1. ■

487 Finally, we will need a lemma about shifting zero bits in and out of the right side of a
 488 value.

489 **Lemma 6.3** (*shl-shr*). *For all $0 < x < 2^{n+2}$,*

$$(while\ (not \circ test\ (n + 1))\ shl \circ while\ even\ shr)\ x = while\ (not \circ test\ (n + 1))\ shl\ x.$$

490 **Proof** Intuitively, this says that if we first shift out all the zero bits and then left shift until
 491 bit $n + 1$ is set, we could get the same result by forgetting about the right shifts entirely;
 492 shifting out zero bits and then shifting them back in should be the identity.

493 Formally, the proof is by induction on x . If $x = xs \therefore I$ is odd, the equality is immediate
 494 since $while\ even\ shr\ x = x$. Otherwise, if $x = xs \therefore O$, on the left-hand side the O is imme-
 495 diately discarded by *shr*, whereas on the right-hand side $xs \therefore O = shl\ xs$, and the extra
 496 *shl* can be absorbed into the *while* since $xs < 2^{n+1}$. What remains is simply the induction
 497 hypothesis. ■

498 With these lemmas under our belt, let's see how to move around a Fenwick array in
 499 order to implement *update* and *query*; we'll begin with *update*. When implementing the
 500 *update* operation, we need to start at a leaf and follow the path up to the root, updating
 501 all the active nodes along the way. In fact, for any given leaf, its closest active parent is
 502 precisely the node stored in the slot that used to correspond to that leaf (see Figure 13). So
 503 to update index i , we just need to start at index i in the Fenwick array, and then repeatedly
 504 find the closest active parent, updating as we go. Recall that the imperative code for *update*
 505 works this way, apparently finding the closest active parent at each step by adding the LSB
 506 of the current index:

```

    public void update(int i, long delta) {
      for (; i < a.length; i += LSB(i)) a[i] += delta;
507 }

```

508 Let's see how to derive this behaviour.

509 To find the closest active parent of a node under a binary indexing scheme, we first move
 510 up to the immediate parent (by dividing the index by two, i.e. performing a right bit shift);
 511 then continue moving up to the next immediate parent as long as the current node is a right
 512 child (i.e. has an odd index). This yields the definition:

513 *activeParentBinary* :: *Bits* → *Bits*
 514 *activeParentBinary* = *while odd shr* ∘ *shr*

515 This is why we used the slightly strange indexing scheme with the root having index
 516 2—otherwise this definition would not work for any node whose active parent is the root!

517 Now, to derive the corresponding operation on Fenwick indices, we conjugate by con-
 518 version to and from Fenwick indices and compute as follows. To make the computation
 519 easier to read, the portion being rewritten is underlined at each step.

$$\begin{aligned}
& b2f' \, n \circ \text{activeParentBinary} \circ f2b' \, n \\
= & \{ \text{expand definitions} \} \\
& \text{unshift} \, (n + 1) \circ \text{inc} \circ \text{while odd shr} \circ \text{shr} \circ \text{dec} \circ \text{shift} \, (n + 1) \\
= & \{ \text{Lemma 6.2 (while-inc-dec)} \} \\
& \text{unshift} \, (n + 1) \circ \text{while even shr} \circ \text{inc} \circ \text{shr} \circ \text{dec} \circ \text{shift} \, (n + 1) \\
= & \{ \text{Lemma 6.1 (shr-inc-dec); shift} \, (n + 1) \, x \text{ is always odd} \} \\
& \text{unshift} \, (n + 1) \circ \text{while even shr} \circ \text{inc} \circ \text{shr} \circ \text{shift} \, (n + 1) \\
= & \{ \text{Lemma 6.1 (shr-inc-dec)} \} \\
& \text{unshift} \, (n + 1) \circ \text{while even shr} \circ \text{shr} \circ \text{inc} \circ \text{shift} \, (n + 1) \\
= & \{ \text{while even shr} \circ \text{shr} = \text{while even shr on an even input} \} \\
& \text{unshift} \, (n + 1) \circ \text{while even shr} \circ \text{inc} \circ \text{shift} \, (n + 1) \\
= & \{ \text{Definition of unshift} \} \\
& \text{clear} \, (n + 1) \circ \text{while} \, (\text{not} \circ \text{test} \, (n + 1)) \, \text{shl} \circ \text{while even shr} \circ \text{inc} \circ \text{shift} \, (n + 1) \\
= & \{ \text{Lemma 6.3 (shl-shr); definition of shift} \} \\
& \text{clear} \, (n + 1) \circ \text{while} \, (\text{not} \circ \text{test} \, (n + 1)) \, \text{shl} \circ \text{inc} \circ \text{while even shr} \circ \text{set} \, (n + 1)
\end{aligned}$$

520 In the final step, since the input x satisfies $x \leq 2^n$, we have $\text{inc} \circ \text{shift} \, (n + 1) < 2^{n+2}$, so
521 Lemma 6.3 applies.

522 Reading from right to left, the pipeline we have just computed performs the following
523 steps:

- 524 1. Set bit $n + 1$
- 525 2. Shift out consecutive zeros until finding the least significant 1 bit
- 526 3. Increment
- 527 4. Shift zeros back in to bring the most significant bit back to position $n + 1$, then
528 clear it.

529 Intuitively, this does look a lot like adding the LSB! In general, to find the LSB, one
530 must shift through consecutive 0 bits until finding the first 1; the question is how to keep
531 track of how many 0 bits were shifted on the way. The *lsb* function itself keeps track via
532 the recursion stack; after finding the first 1 bit, the recursion stack unwinds and re-snocs
533 all the 0 bits recursed through on the way. The above pipeline represents an alternative
534 approach: set bit $n + 1$ as a “sentinel” to keep track of how much we have shifted; right
535 shift until the first 1 is literally in the ones place, at which point we increment; and then
536 shift all the 0 bits back in by doing left shifts until the sentinel bit gets back to the $n + 1$
537 place. One example of this process is illustrated in Figure 19. Of course, this only works
538 for values that are sufficiently small that the sentinel bit will not be disturbed throughout
539 the operation.

540 To make this more formal, we begin by defining a helper function *atLSB*, which does
541 an operation “at the LSB”, that is, it shifts out 0 bits until finding a 1, applies the given
542 function, then restores the 0 bits.

543 $\text{atLSB} :: (\text{Bits} \rightarrow \text{Bits}) \rightarrow \text{Bits} \rightarrow \text{Bits}$
544 $\text{atLSB} _ (\text{Rep } O) = \text{Rep } O$
545 $\text{atLSB } f \, (\text{bs} :: O) = \text{atLSB } f \, \text{bs} :: O$
546 $\text{atLSB } f \, \text{bs} = f \, \text{bs}$

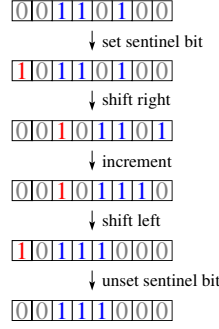


Fig. 19. Adding LSB with a sentinel bit + shifts.

547 **Lemma 6.4** (add-lsb). *For all $x :: \text{Bits}$, $x + \text{lsb } x = \text{atLSB inc } x$ and $x - \text{lsb } x = \text{atLSB dec } x$.*

548 **Proof** Straightforward induction on x . ■

549 We can formally relate the “shifting with a sentinel” scheme to the use of *atLSB*, with
 550 the following (admittedly rather technical) lemma:

551 **Lemma 6.5** (sentinel). *Let $n \geq 1$ and let $f :: \text{Bits} \rightarrow \text{Bits}$ be a function such that*

- 552 1. *$(f \circ \text{set } (n + 1)) x = (\text{set } (n + 1) \circ f) x$ for any $0 < x < 2^n$, and*
 553 2. *$f x < 2^{n+1}$ for any $0 < x < 2^n + 2^{n-1}$.*

554 *Then for all $0 < x < 2^n$,*

$$(\text{unshift } (n + 1) \circ f \circ \text{shift } (n + 1)) x = \text{atLSB } f x.$$

555 The proof is rather tedious and not all that illuminating, so we omit it (an extended
 556 version including a full proof may be found on the author’s website, at [http://ozark.
 557 hendrix.edu/~yorgey/pub/Fenwick-ext.pdf](http://ozark.hendrix.edu/~yorgey/pub/Fenwick-ext.pdf)). However, we do note that both *inc*
 558 and *dec* fit the criteria for f : incrementing or decrementing some $0 < x < 2^n$ cannot affect
 559 the $(n + 1)$ st bit as long as $n \geq 1$, and the result of incrementing or decrementing a number
 560 less than $2^n + 2^{n-1}$ will be a number less than 2^{n+1} . We can now put all the pieces together
 561 show that adding the LSB at each step is the correct way to implement *update*.

562 **Theorem 6.6.** *Adding the LSB is the correct way to move up a Fenwick-indexed tree to*
 563 *the nearest active parent, that is,*

$$\text{activeParentFenwick} = \text{b2f}' n \circ \text{activeParentBinary} \circ \text{f2b}' n = \lambda x \rightarrow x + \text{lsb } x$$

564 *everywhere on the range $[1, 2^n)$. (We exclude 2^n since it corresponds to the root of the tree*
 565 *under a Fenwick indexing scheme.)*

566 **Proof**

$$\begin{aligned}
 & \text{b2f}' n \circ \text{activeParentBinary} \circ \text{f2b}' n \\
 = & \{ \text{Previous calculation} \} \\
 & \text{unshift } (n + 1) \circ \text{inc} \circ \text{shift } (n + 1)
 \end{aligned}$$

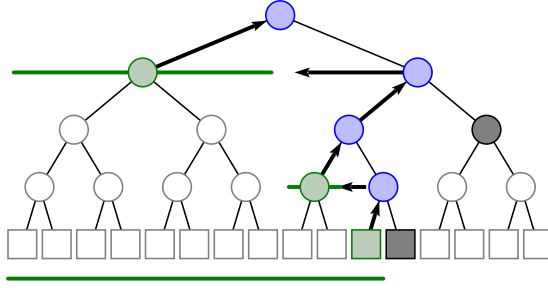


Fig. 20. Moving up a segment tree to find successive prefix segments.

$$\begin{aligned}
 &= \{ \text{Lemma 6.5 (sentinel)} \} \\
 &\text{atLSB inc} \\
 &= \{ \text{Lemma 6.4 (add-lsb)} \}
 \end{aligned}$$

567 $\lambda x \rightarrow x + \text{lsb } x$ ■

568 We can carry out a similar process to derive an implementation for prefix query (which
 569 supposedly involves *subtracting* the LSB). Again, if we want to compute the sum of $[1, j]$,
 570 we can start at index j in the Fenwick array, which stores the sum of the unique segment
 571 ending at j . If the node at index j stores the segment $[i, j]$, we next need to find the unique
 572 node storing a segment that ends at $i - 1$. We can do this repeatedly, adding up segments
 573 as we go.

574 Staring at Figure 20 for inspiration, we can see that what we want to do is find the
 575 *left sibling* of our *closest inactive parent*, that is, we go up until finding the first ancestor
 576 which is a right child, then go to its left sibling. Under a binary indexing scheme, this can
 577 be implemented simply as:

578 $\text{prevSegmentBinary} :: \text{Bits} \rightarrow \text{Bits}$
 579 $\text{prevSegmentBinary} = \text{dec} \circ \text{while even shr}$

580 **Theorem 6.7.** *Subtracting the LSB is the correct way to move up a Fenwick-indexed tree*
 581 *to the active node covering the segment previous to the current one, that is,*

$$\text{prevSegmentFenwick} = \text{b2f}' \circ \text{prevSegmentBinary} \circ \text{f2b}' \circ \lambda x \rightarrow x - \text{lsb } x$$

582 *everywhere on the range $[1, 2^n)$.*

583 **Proof**

$$\begin{aligned}
 &\text{b2f}' \circ \text{prevSegmentBinary} \circ \text{f2b}' \circ \lambda x \rightarrow x - \text{lsb } x \\
 &= \{ \text{expand definitions} \} \\
 &\text{unshift } (n + 1) \circ \text{inc} \circ \text{dec} \circ \text{while even shr} \circ \text{dec} \circ \text{shift } (n + 1) \\
 &= \{ \text{inc} \circ \text{dec} = \text{id} \} \\
 &\text{unshift } (n + 1) \circ \text{while even shr} \circ \text{dec} \circ \text{shift } (n + 1) \\
 &= \{ \text{Definition of unshift} \} \\
 &\text{clear } (n + 1) \circ \text{while } (\text{not} \circ \text{test } (n + 1)) \text{ shl} \circ \text{while even shr} \circ \text{dec} \circ \text{shift } (n + 1)
 \end{aligned}$$

$$\begin{aligned}
&= \frac{\{ \text{Lemma 6.3 (shl-shr)} \}}{\text{clear } (n+1) \circ \text{while } (\text{not} \circ \text{test } (n+1)) \text{ shl} \circ \text{dec} \circ \text{shift } (n+1)} \\
&= \frac{\{ \text{Definition of unshift} \}}{\text{unshift } (n+1) \circ \text{dec} \circ \text{shift } (n+1)} \\
&= \frac{\{ \text{Lemma 6.5 (sentinel)} \}}{\text{atLSB dec}} \\
&= \frac{\{ \text{Lemma 6.4 (add-lsb)} \}}{\lambda x \rightarrow x - \text{lsb } x}
\end{aligned}$$

584

■

585 7 Conclusion

586 Historically, to my knowledge, Fenwick trees were not actually developed as an optimiza-
587 tion of segment trees as presented here. This has merely been a fictional—but hopefully
588 illuminating—alternate history of ideas, highlighting the power of functional thinking,
589 domain-specific languages, and equational reasoning to explore relationships between dif-
590 ferent structures and algorithms. As future work, it would be interesting to explore some of
591 the mentioned generalizations of segment trees, to see whether one can derive Fenwick-like
592 structures that support additional operations.

593 Acknowledgements

594 Thanks to the anonymous JFP reviewers for their helpful feedback, which resulted in a
595 much improved presentation. Thanks also to Penn PL Club for the opportunity to present
596 an early version of this work.

597 Conflicts of Interest

598 None.

599 References

- 600 Apfelmus, H. (2009) Monoids and finger trees. [https://apfelmus.nfshost.com/articles/](https://apfelmus.nfshost.com/articles/monoid-fingertree.html)
601 [monoid-fingertree.html](https://apfelmus.nfshost.com/articles/monoid-fingertree.html). [Online; accessed 10-Jun-2024].
602 Bird, R. & Gibbons, J. (2002) Arithmetic coding with folds and unfolds. In *International School on*
603 *Advanced Functional Programming*. Springer, pp. 1–26.
604 Claessen, K. & Hughes, J. (2000) Quickcheck: A lightweight tool for random testing of haskell
605 programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional*
606 *Programming (ICFP)*, pp. 268–279.
607 Erwig, M. & Jones, S. P. (2001) Pattern guards and transformational patterns. *Electron. Notes*
608 *Theoret. Comput. Sci.* **41**(1), 3.
609 Fenwick, P. M. (1994) A new data structure for cumulative frequency tables. *Software Pract. Exper.*
610 **24**(3), 327–336.
611 Halim, S., Halim, F. & Effendy, S. (2020) *Competitive Programming 4: The Lower Bound of*
612 *Programming Contests in the 2020s*. Lulu Press.

- 613 Hinze, R. & Paterson, R. (2006) Finger trees: A simple general-purpose data structure. *J. Funct.*
614 *Program.* **16**(2), 197–217.
- 615 Ivanov, M. (2011a) Fenwick tree. [https://cp-algorithms.com/data_structures/fenwick.](https://cp-algorithms.com/data_structures/fenwick.html)
616 [html](https://cp-algorithms.com/data_structures/fenwick.html). [Online; accessed 21-Nov-2024].
- 617 Ivanov, M. (2011b) Segment tree. [https://cp-algorithms.com/data_structures/segment_](https://cp-algorithms.com/data_structures/segment_tree.html)
618 [tree.html](https://cp-algorithms.com/data_structures/segment_tree.html). [Online; accessed 03-Jun-2024].
- 619 Pickering, M., Érdi, G., Peyton Jones, S. & Eisenberg, R. A. (2016) Pattern synonyms. In
620 Proceedings of the 9th ACM SIGPLAN International Symposium on Haskell, pp. 80–91.
- 621 Rissanen, J. & Langdon, G. G. (1979) Arithmetic coding. *IBM J. Res. Dev.* **23**(2), 149–162.
- 622 Ryabko, B. Y. (1989) A fast on-line code. *Doklady Akademii Nauk. Russian Academy of Sciences*,
623 pp. 548–552.
- 624 Wikipedia Contributors. (2024) Segment tree — Wikipedia, the free encyclopedia. [https://en.](https://en.wikipedia.org/wiki/Segment_tree)
625 [wikipedia.org/wiki/Segment_tree](https://en.wikipedia.org/wiki/Segment_tree). [Online; accessed 03-Jun-2024].